MPRD Security Whitepaper
Verifiable Decision Rails for Tool-Using AI

MPRD Project

Community Draft v0.3, February 2026

Abstract

MPRD (*Model Proposes, Rules Decide*) is a security architecture for deploying tool-using AI systems behind a small, deterministic, auditable *decision rail*. The model is treated as fully untrusted: it may propose arbitrary actions, but it cannot execute. The only path to side effects is a trusted executor that enforces an explicit policy predicate and (optionally) a proof-carrying transcript. The core goal is a crisp safety property: *no unauthorized action executes*, even if the model is buggy or adversarial.

Status. This document is a Community Draft intended for public consumption and community review. It describes MPRD at the level of architecture, invariants, and verification rails, without relying on proprietary model internals.

1 Executive summary

The problem. Modern AI systems increasingly interact with external tools: APIs, code execution, wallets, databases, and infrastructure controls. These actions are *real-world side effects*. If the model is compromised, misprompted, or simply wrong, the damage can be immediate and irreversible.

The key idea. Treat the model as a powerful *proposal generator*, not as a decision maker. Put all side effects behind a deterministic, policy-driven boundary.

What MPRD provides.

- A clean separation of roles: **propose** (untrusted) vs **decide** (trusted) vs **execute** (trusted).
- An explicit, auditable policy predicate Allowed(·) for every trust-boundary action.
- A safety invariant: executed actions are always allowed by policy.
- A pathway to third-party verifiability using proof-carrying transcripts (e.g., ZK receipts).

Key clarification. In MPRD, the model is not the agent and is not the executor. The model provides intelligence by proposing candidates. Agency is concentrated in the executor and constrained by explicit policy.

What MPRD does not provide by itself.

- **Policy correctness:** MPRD enforces whatever policy is installed, even if flawed.
- **Liveness:** safety can permit safe stalls; liveness requires additional control-plane mechanisms.
- **Model goodness:** MPRD reduces blast radius but does not "fix" the model.

2 Models, agents, and executors (terminology)

Security discussions around "agents" often blur together three very different things: the model, the agent, and the executor. MPRD is primarily about separating them.

Model. In this whitepaper, a *model* is an inference engine (a learned function) that maps inputs to outputs: text, code, plans, or candidate actions. It can be extremely capable, but it is not assumed to be trustworthy. In MPRD, the model occupies the *proposer* role and is treated as an untrusted source of intelligence for the rail.

Agent. An *agent* is the *whole system* that produces side effects in the world: model + memory + prompting + retrieval + planning + tool adapters + decision logic + scheduling + execution. In typical "tool-using agent" designs, the model directly emits tool calls and the runtime executes them. In that design, model output becomes de facto authority.

Executor. An *executor* is the minimal, trusted boundary component that has the ability to cause side effects (network calls, filesystem writes, funds transfers, infrastructure changes, etc.). In MPRD, the executor is *not* "a model with tools"; it is a deterministic gatekeeper that performs side effects only when an explicit policy authorizes them.

Decision rail. The *decision rail* is the trusted pipeline between proposals and side effects: policy evaluation, selection, transcript construction, verification, and execution. The rail is designed to be deterministic, auditable, and fail-closed.

3 Separating intelligence from agency

The core claim behind MPRD is that *intelligence and agency should not be coupled.*

Intelligence. In practice, what we want from a model is *search power*: the ability to generate high-quality candidate actions and plans from context. This can be obtained from many sources: foundation models, ensembles, search algorithms, heuristics, or even human input.

Agency. Agency is the ability to *do* things in the world. Agency is high-stakes because it crosses trust boundaries. MPRD makes agency explicit and scarce: only the executor has it, and the executor is governed.

The separation. MPRD forces all intelligence to pass through a narrow interface:

> The model may propose. The system decides. The executor enacts.

This turns model output from "commands" into *suggestions*, and it turns side effects into a *permissioned* operation that must be justified by explicit policy and (optionally) verifiable evidence.

Neuro-symbolic view. MPRD can be viewed as a neuro-symbolic architecture pattern. Learned models provide the neural component by generating proposals, while the decision rail provides the symbolic component by enforcing explicit, deterministic rules and checks at the side-effect boundary.

How intelligence is extracted and commodified

By separating intelligence (proposal generation) from authority (execution), MPRD makes the proposer *replaceable*:

- You can swap model providers without changing the security story.
- You can run multiple models and union their candidates, while keeping one rules engine and one executor.
- You can improve proposal quality (better candidates) without granting additional execution authority or expanding the trusted computing base.

This factorization treats proposal generation as an interchangeable upstream service. The governed rail consumes only a bounded candidate set, and the safety boundary does not depend on which model produced it.

Swarm proposing (many models, one rail)

MPRD is compatible with a single proposer or a *swarm* of proposers. Multiple models can run in parallel, each producing candidate actions and supporting evidence. Their outputs are merged into a single candidate set (with deterministic de-duplication and ordering), and then the same decision rail evaluates policy and selects a committed action.

The important point is that adding more "intelligence sources" does not add more authority. A swarm can improve candidate quality and coverage, but it cannot bypass the executor or the policy predicate.

4 The architecture

MPRD separates a tool-using AI system into three roles:

- **Proposer(s) (untrusted):** given a state, produce a bounded set of candidate actions (one model or many).
- **Rules engine (trusted, deterministic):** evaluates which actions are allowed under a versioned policy.
- **Executor (trusted):** performs side effects *only* for actions authorized by the rail.

The decision rail is a security boundary. It is intentionally small, pure, and deterministic so it can be audited, tested, fuzzed, and mechanically specified.

Conceptual data flow

```
state s
  ↓
proposer(s) produce candidates C = [a1, a2, ...]
```

```
  ↓
rules evaluate Allowed(policy, s, ai) for each ai
  ↓
selector chooses a* with Allowed(policy, s, a*) = true
  ↓
token/proof bind (policy, s, C, a*) to a verifiable transcript
  ↓
executor executes a* only if transcript verifies (fail-closed)
```

4.1 A more explicit decomposition of "agent"

To make the separation concrete, it helps to break an "agent" into subcomponents:

- **Proposer(s):** learned models, retrieval, planning, and search (untrusted, neural).
- **Canonicalizers:** deterministic encodings and hashes for state/actions/candidates (trusted, pure).
- **Policy evaluation:** explicit Allowed predicate and selection logic (trusted, pure).
- **Attestation (optional):** generation of a proof-carrying transcript (trusted, deterministic).
- **Verification:** deterministic, fail-closed validation of the transcript (trusted, pure).
- **Executor:** performs the single committed side effect (trusted, effectful).

Only the last component is allowed to touch the outside world.

4.2 Design principles

MPRD is guided by a small set of security principles:

- **Capability separation:** the model has *zero* direct execution capability.
- **Small trusted core:** keep the rules/verifier/executor boundary minimal and reviewable.
- **Determinism:** canonicalization, hashing, tie-breaking, and verification must be stable.
- **Fail-closed:** unknown versions, missing signals, parse errors, and proof failures reject execution.
- **Explicitness:** "what is allowed" must be a first-class predicate, not implicit conventions.
- **Auditability:** decisions should produce structured artifacts (hashes/receipts/logs) suitable for review.

5 Threat model

Adversary. MPRD treats the proposer (the model) as fully untrusted: it may be misaligned, compromised, manipulated by prompt injection, or maliciously instructed. MPRD also assumes the environment between proposer and executor is hostile: inputs can be tampered with, messages replayed, and operators can attempt to route around checks.

Trusted computing base (TCB). The trusted core is the rules engine, verifier, and executor boundary code. This boundary should be small, deterministic, and hardened. If cryptography is used, standard assumptions apply (hashes/signatures are unbroken).

Representative threats and mitigations.

Threat	Mitigation pattern in MPRD
Model proposes an unsafe action	Rules engine rejects; executor cannot run without authorization.
Prompt injection / data poisoning	Treat proposer output as untrusted; require explicit allow predicates and verifiable transcripts.
Replay of a previously valid action	Bind execution to a nonce/tx hash; single-use consumption; reject duplicates.
Downgrade to a weaker verification path	Version/ID allowlists; fail-closed on unknown kinds/versions; explicit production profiles.
Tampering with policy or routing to a different verifier	Policy identity bound (hash/epoch/root); allowlisted guest images; registry-bound verification.
DoS via oversized artifacts	Bounded parsing everywhere; strict size limits at ingress; deterministic decoding.

Out of scope. MPRD does not prevent vulnerabilities outside the executor boundary (e.g., unintended direct tool access), and it does not solve covert channels. It also does not guarantee the policy is morally or operationally correct.

6 Related work

MPRD is a security architecture for enforcing explicit authorization and integrity constraints at a narrow side-effect boundary. This is closely related to classic security principles (least privilege, complete mediation) and their practical enforcement via reference monitors and policy enforcement mechanisms [1, 2].

The "proof-carrying" framing aligns with proof-carrying code and related approaches, where untrusted computation is accompanied by evidence that a small checker can validate [3]. When the evidence is a succinct cryptographic proof, the approach connects to verifiable computation and zero-knowledge proof systems [4].

From the perspective of AI safety and formal methods, MPRD is related to runtime enforcement and shielding approaches that constrain an agent's actions to satisfy a safety specification [5]. MPRD differs mainly in emphasis: the policy predicate is treated as an explicit governance artifact, and the executor boundary is designed for auditable, fail-closed verification.

7 Core invariants and contracts

MPRD is not just "one invariant." It is a small set of invariants that together separate intelligence (proposals) from authority (execution).

7.1 Invariant summary (high level)

Invariant	Statement (high level)	Typical enforcement
Authorization (safety)	Executed actions are allowed by policy.	Policy evaluation + executor gate
Binding	The executed action matches the verified transcript.	Hash commitments + verification
Anti-replay	A valid decision cannot be used twice.	Nonce/tx binding + one-time consume
Fail-closed versioning	Unknown/missing/invalid inputs reject.	Strict decoders + allowlists
Determinism	Same inputs yield the same verdict and hashes.	Canonicalization + stable tie-breakers
Boundedness	Untrusted inputs and execution are resource-bounded.	Size limits + fuel/time caps

7.2 Minimal formal model

We use a minimal abstract model to state what is proved and what is assumed.

Definition 1 (Types)**.** *Let S be a set of states, A a set of actions, and P a set of policies. Let $\mathcal{L}(A)$ denote finite lists over A.*

Definition 2 (Components)**.** *We model the decision rail by the following functions and predicates:*

- ***Proposer:*** $M : S \to \mathcal{L}(A)$ *(implemented by one or more learned models and treated as untrusted).*
- ***Authorization predicate:*** $\text{Allowed} : P \times S \times A \to \{\mathit{true}, \mathit{false}\}$.
- ***Selector:*** $\text{Sel} : P \times S \times \mathcal{L}(A) \to A$.
- ***Execution event:*** $\text{ExecCalled}(p, s, a)$ *denotes that the executor is invoked under policy p, state s, with action a.*

Assumption 1 (Selector contract)**.** *For all $p \in P$, $s \in S$, and $C \in \mathcal{L}(A)$,*

$$\text{Sel}(p, s, C) = a \;\Rightarrow\; a \in C \wedge \text{Allowed}(p, s, a) = \mathit{true}.$$

Assumption 2 (Execution guard)**.** *For all $p \in P$, $s \in S$, and $a \in A$,*

$$\text{ExecCalled}(p, s, a) \;\Rightarrow\; \exists C.\, C = M(s) \wedge a = \text{Sel}(p, s, C).$$

7.3 Authorization theorem (safety and security)

The central statement is both a safety property (in the formal methods sense) and an authorization and integrity property (in the security sense):

$$\boxed{\forall \text{ executed actions } a\text{: } \text{Allowed}(\text{policy}, \text{state}, a) = \text{true}.}$$

Theorem 1 (Authorization safety property)**.** *Under the Selector contract and Execution guard, for all $p \in P$, $s \in S$, and $a \in A$,*

$$\text{ExecCalled}(p, s, a) \Rightarrow \text{Allowed}(p, s, a) = \mathit{true}.$$

Proof sketch. Fix p, s, a and assume ExecCalled(p, s, a). By the Execution guard, there exists C such that $C = M(s)$ and $a = \text{Sel}(p, s, C)$. By the Selector contract, $\text{Sel}(p, s, C) = a$ implies $\text{Allowed}(p, s, a) = \text{true}$. Combining the implications yields the claim. □

The theorem says: regardless of how the proposer behaves internally, the system never executes an action that violates the explicit authorization predicate.

7.4 Binding, anti-replay, and determinism (why the invariant is enforceable)

To make the safety story real in hostile environments, MPRD relies on additional invariants:

- **Binding:** the policy/state/candidate set and the chosen action are committed (e.g., by hashes/IDs), and the executor checks the committed values before executing.
- **Anti-replay:** decisions carry a nonce/tx hash that is consumed exactly once; replays are rejected.
- **Fail-closed:** unknown versions, missing signals, parse errors, and unverifiable proofs reject execution.
- **Determinism:** canonicalization, hashing, and selection use stable ordering and explicit tie-breaking so verifiers can be strict.
- **Boundedness:** all untrusted inputs are size-bounded, and execution is constrained by explicit resource limits.

These supporting invariants prevent confused-deputy and transcript-mismatch failure modes, where a system verifies one set of commitments but executes a different action.

7.5 Mechanized proof artifact (Lean)

In this repository, the authorization invariant is mechanized in Lean 4 [6] as an abstract theorem about the pipeline structure (not about any particular model internals):

- Lean artifact: `proofs/lean/MPRD_Theorem.lean`
- Build: `cd proofs/lean && lake build`

What the proof assumes. Like most architecture-level proofs, it relies on contracts that must be enforced by the implementation:

- **Selector contract:** the chosen action comes from the candidate set and is allowed.
- **Execution guard:** the executor runs only on outputs that have passed verification.

In production deployments, MPRD aims to make these contracts concrete via proof-carrying transcripts, bounded parsing, strict allowlists, and fail-closed verification.

8 The executor (the side-effect boundary)

The executor is the component that makes the separation between intelligence and agency concrete. It is the *only* part of the system allowed to touch the outside world.

8.1 Why the executor must be explicit

If a model runtime can directly call tools (HTTP, shell, wallets, databases), then model outputs are effectively *authority*. In that regime, "safety" becomes a best-effort property of prompts and model behavior. MPRD instead insists on an explicit, minimal executor boundary that can be audited and governed.

8.2 Executor responsibilities

An MPRD executor should be viewed as a hardened transaction processor, not as an "agent loop." At a minimum, it must:

- **Decode and validate the transcript (fail-closed):** reject unknown versions, malformed encodings, and oversized inputs.
- **Verify evidence:** check signatures and/or proof artifacts when operating in trustless modes.
- **Enforce binding:** ensure the executed action is *exactly* the one committed in the verified transcript.
- **Enforce authorization context:** ensure the policy identity and epoch/root are authorized under governance.
- **Prevent replay:** consume a nonce/tx hash exactly once (reject duplicates).
- **Enforce resource bounds:** timeouts, max bytes, rate limits, and explicit fuel semantics where applicable.
- **Execute one committed effect:** perform the side effect through a narrow, typed adapter interface.
- **Emit audit artifacts:** record what was verified and what was executed, with stable IDs for later review.

8.3 What an executor is not

If the executor contains dynamic interpretation of model-provided code, it is no longer a security boundary. Anti-patterns include:

- executing arbitrary scripts produced by the model,
- allowing tool routing based on untrusted hints without allowlists,
- accepting "partial" verification results (e.g., timeouts) as success, and
- silently falling back to weaker verification modes under error conditions.

8.4 Executor algorithm (illustrative)

The executor typically follows a two-phase shape: *verify, then execute*, with strict checks and explicit tie points. An illustrative skeleton is:

```
execute(bundle):
  # 1) Decode transcript (fail-closed)
  t = decode_transcript(bundle)
  require known_versions(t)
  require size_bounded(t)

  # 2) Verify cryptographic evidence (optional but recommended)
  require verify_proof_or_sig(t.evidence, t.commitments)

  # 3) Enforce authorization context
  require policy_authorized(t.policy_id, t.policy_epoch, t.registry_root)

  # 4) Enforce binding to state/actions/candidates
  require hash(canon_state(t.state))          == t.state_hash
  require hash(canon_candidates(t.candidates)) == t.candidate_set_hash
  require hash(canon_action(t.action))       == t.action_hash

  # 5) Anti-replay
  require consume_once(t.nonce_or_tx_hash)

  # 6) Resource bounds (inputs + execution)
  require limits_ok(t.limits)

  # 7) Execute the single committed action through adapters
  return run_action_via_allowlisted_adapter(t.action)
```

Key point. The proposer(s) are upstream of this boundary and can be arbitrary. The executor is downstream, deterministic, and fail-closed.

9 Verification rails (how contracts become enforceable)

The practical goal is to reduce "trust" in the operator or model to a verifiable, replayable check. MPRD does this with layered rails; deployments can adopt them incrementally.

9.1 Candidate aggregation and deterministic selection

In MPRD, the proposer stage may be a single model, a search procedure, or a swarm of heterogeneous models. Regardless of how candidates are produced, the rail treats them as untrusted data and brings them into a *canonical form*:

- deterministically normalize and de-duplicate candidates,
- commit to the candidate set by a stable hash/ID, and
- select an allowed action using explicit, deterministic tie-breaking.

This is a key part of the separation and enables commoditization of proposal generation: proposals can be collected from many sources, but only a committed, policy-justified result is allowed to reach the executor.

9.2 Anti-replay and idempotence

Any real executor needs an "exactly-once" story. MPRD typically binds each decision to a nonce or transaction hash and requires:

- **single-use consumption:** the nonce is accepted at most once, and
- **replay resistance:** previously executed decisions cannot be re-submitted to produce the same effect twice.

9.3 Registry-bound verification and allowlists

In trustless or third-party-verifiable deployments, the verifier must not rely on untrusted routing hints. Instead, MPRD emphasizes:

- **policy identity binding:** actions are authorized under a specific policy hash and authorization context (epoch/root),
- **allowlisted execution kinds/versions:** unknown IDs are rejected, and
- **allowlisted verifier images/artifacts:** execution is bound to an auditable allowlist (e.g., a signed manifest).

This prevents downgrade and substitution attacks where an attacker tries to route verification through a weaker path.

9.4 Policy rail (explicit predicates)

The Allowed predicate is treated as a first-class, versioned artifact: reviewed, change-controlled, and evaluated deterministically. This repository includes:

- Tau policies and governance rails: `policies/` and `docs/POLICY_ALGEBRA.md`
- Change-safety tooling (semantic hashes + counterexample diffs): `docs/POLICY_CERTIFICATION.md`

Why this matters. In many systems, "policy" is scattered across code paths and defaults. MPRD instead makes authorization explicit and auditable: if a transition can happen, there is a predicate authorizing it.

9.5 Policy certification (semantic diffs, not just syntax)

Policy changes are a common source of silent regressions: refactors and rewrites that look harmless but alter meaning. MPRD includes tooling to compare policies by semantics and, when they differ, produce a concrete counterexample assignment. This supports a workflow where policy upgrades are reviewable, testable, and explainable.

See `docs/POLICY_CERTIFICATION.md`.

9.6 Proof-carrying rail (optional, for trustless verification)

For untrusted deployments, MPRD can attach a proof that binds the decision to a small, versioned transcript (token + journal/receipt). The verifier checks this transcript deterministically and fail-closed.

In ZK-backed modes, the attestation commits to (among other fields):

- governed policy identity (hash/epoch/root),
- state identity and provenance commitments,
- candidate set and chosen action commitments,
- resource limits / fuel semantics,
- anti-replay binding (nonce/tx hash),
- schema and version IDs (unknown versions rejected).

See `docs/PRODUCTION_ZK.md` for the recommended production verifier pattern (registry-bound verification and allowlisted guest images).

9.7 Determinism and canonicalization

Security checks often fail in the corners: ambiguous encodings, unstable hashes, or nondeterministic tie-breaking. MPRD emphasizes canonicalization and stable IDs so that:

- the same inputs produce the same commitments,
- verifiers can be strict (fail-closed) without accidental false rejects,
- audits are reproducible, and
- semantic "diffs" can produce concrete counterexamples.

9.8 Bounded parsing and resource limits

The executor boundary is a natural DoS target: attackers will try to send huge policies, states, receipts, or candidate sets. MPRD deployments therefore treat *boundedness* as a first-class invariant:

- all untrusted inputs are size-bounded and decoded with fail-closed parsers, and
- execution is constrained by explicit limits (timeouts, max bytes, and/or a fuel model).

Without this, even a perfectly correct policy predicate can be rendered unusable by resource exhaustion.

10 Deployment guidance (high level)

Different deployments can place the trust boundary at different points; MPRD supports both local and trustless operation. At a high level:

- **Dev/local:** use the rail architecture to harden the boundary even when proofs are not required.

- **Production/trustless:** require proof-carrying transcripts and strict allowlists at the verifier boundary.
- **Privacy-required:** keep transcripts verifiable while hiding sensitive inputs where needed (deployment-specific).

Common pitfalls. The most common ways to break the security story are:

- **Bypass channels:** any way the model can cause side effects without the executor boundary.
- **Downgrades:** falling back to non-verified execution paths under load or failure.
- **Unbounded inputs:** parsing receipts, policies, or states without explicit size limits.
- **Replay/TOCTOU:** executing with stale state or reusing old tokens.
- **Silent nondeterminism:** unstable hashes, inconsistent ordering, or platform-dependent behavior.

11 Community draft process

This whitepaper is a Community Draft. The version (v0.3) is intended to make review and discussion concrete. Helpful feedback focuses on:

- whether the separation between proposer(s), rules, and executor is described clearly,
- whether the executor boundary is explained in a way that maps to real deployments,
- whether the invariants listed here match the failure modes practitioners see in tool-using systems, and
- which examples or diagrams would make the rails easier to understand.

12 Hardening checklist

MPRD is an architecture, not a complete security solution. Secure deployments still require careful implementation and operational discipline. Use the repository checklist as a release gate:

- `docs/SECURITY_HARDENING_CHECKLIST.md`

Minimum security bar (recommended). For production-like deployments, reviewers should insist on:

- strict allowlists for execution kinds/versions and guest image IDs,
- versioned, fail-closed transcript decoding,
- anti-replay enforcement bound into the statement,
- bounded parsing for all untrusted inputs, and
- deterministic canonicalization/hashing with reproducible builds.

13 Limitations and open problems

MPRD provides a strong *safety* boundary, but important problems remain:

- **Policy correctness:** the rail enforces the policy in force, even if flawed or incomplete.
- **Liveness:** the architecture can safely reject everything; progress requires additional mechanisms.
- **Refinement fidelity:** proving the full implementation refines the abstract model is a larger effort.
- **Side channels:** covert channels and unintended I/O must be treated as part of the executor boundary.

14 How to verify artifacts in this repository

Lean proof.

```
cd proofs/lean && lake build
```

ZK deployment guidance.

```
docs/PRODUCTION_ZK.md
```

Policy rails and certification tooling.

```
docs/POLICY_ALGEBRA.md
docs/POLICY_CERTIFICATION.md
```

Further reading. `README.md` (overview), `docs/PRODUCTION_READINESS.md`, and `docs/PRODUCTION_ZK.md`.

References

[1] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, 1975.

[2] F. B. Schneider, "Enforceable Security Policies," *ACM Transactions on Information and System Security*, 2000.

[3] G. C. Necula, "Proof-Carrying Code," in *Proc. 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1997.

[4] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza, "Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture," in *Proc. 23rd USENIX Security Symposium*, 2014.

[5] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe Reinforcement Learning via Shielding," in *Proc. AAAI Conference on Artificial Intelligence (AAAI)*, 2018. Preprint: arXiv:1708.08611.

[6] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *Proc. 28th International Conference on Automated Deduction (CADE-28)*, Springer, 2021.